

AMS 598 Final Exam Lecture Notes

MapReduce, ADMM, Logistic Regression, SVM, Random Forests

Contents

1	MapReduce Algorithms	2
1.1	The MapReduce Paradigm	2
1.2	TF-IDF in MapReduce	2
1.3	Relational Algebra Operations in MapReduce	4
1.4	Matrix Multiplication in MapReduce	6
1.5	PageRank	7
2	Penalty-Based Methods and ADMM	8
2.1	Ridge Regression (L2 Penalty)	8
2.2	LASSO (L1 Penalty)	9
2.3	Penalized Logistic Regression	9
2.4	ADMM: Alternating Direction Method of Multipliers	9
2.5	Consensus Optimization via ADMM	10
2.6	ADMM for Logistic Regression with a Custom Penalty Term	11
3	Support Vector Machines (SVM)	14
3.1	Linear Separating Hyperplane	14
3.2	Margin and Optimal Separator	15
3.3	Hard-Margin SVM	15
3.4	Soft-Margin SVM	15
3.5	Dual Formulation and Support Vectors	15
3.6	Nonlinear SVM and the Kernel Trick	16
4	Random Forests	16
4.1	Decision Trees (CART)	17
4.2	Random Forest Algorithm	17
4.3	Out-of-Bag (OOB) Error	18
4.4	Variable Importance in Random Forests	18
4.5	Random Forests for Big Data	18

1 MapReduce Algorithms

1.1 The MapReduce Paradigm

MapReduce is a programming model for large-scale data processing on a cluster.

- Input data is split into *key-value pairs* (k, v) .
- **Map phase:** Apply a map function to each input pair, producing intermediate key-value pairs:

$$(k, v) \mapsto \{(k_1, v_1), (k_2, v_2), \dots\}.$$

- The system groups all intermediate values by key.
- **Reduce phase:** For each key k , apply a reduce function to the list of values:

$$(k, [v_1, v_2, \dots]) \mapsto (k, v_{\text{out}}).$$

- Tasks:
 - Map tasks run in parallel on different data chunks.
 - Reduce tasks run in parallel on different keys.

Key idea: Make each map and reduce operation independent so that they can be parallelized.

1.2 TF-IDF in MapReduce

TF-IDF: Term Frequency-Inverse Document Frequency.

Motivation

We have a collection of N documents. For a term i and a document j :

- f_{ij} : frequency (count) of term i in document j .
- n_i : number of documents in which term i appears.

Definitions:

$$\text{TF}_{ij} = \frac{f_{ij}}{\max_k f_{kj}}, \quad \text{IDF}_i = \log_2 \frac{N}{n_i}, \quad \text{TFIDF}_{ij} = \text{TF}_{ij} \cdot \text{IDF}_i.$$

Goals:

- Down-weight *very frequent* terms (e.g. “the”).
- Up-weight terms that are concentrated in relatively few documents and thus more indicative of topic.

Information Needed

To compute TF-IDF, we need:

- (1) Number of times term i appears in each document j (f_{ij}).
- (2) Maximum frequency in each document j (i.e. $\max_k f_{kj}$).
- (3) Number of documents containing term i (n_i).
- (4) Total number of documents N .

This naturally leads to a multi-pass MapReduce pipeline.

Job 1: Word Frequency in Each Document

Input: (docname, contents) pairs.

Mapper:

- For each word $\mathbf{w}$ in document d , emit:

$$(d, \text{contents}) \mapsto ((\mathbf{w}, d), 1).$$

Reducer:

- For each key $(\mathbf{w}, d)$, sum the counts:

$$((\mathbf{w}, d), [1, 1, \dots, 1]) \mapsto ((\mathbf{w}, d), f_{wd}).$$

Output: $((\mathbf{w}, d), f_{wd})$.

Job 2: Max Word Count per Document

Input: $((\mathbf{w}, d), f_{wd})$ from Job 1.

Mapper:

$$((\mathbf{w}, d), f_{wd}) \mapsto (d, (\mathbf{w}, f_{wd})).$$

Reducer:

- For each document d , we receive $(\mathbf{w}, f_{wd})$ for many words.
- Compute $m_d = \max_{\mathbf{w}} f_{wd}$.
- For each word in this doc, re-emit:

$$((\mathbf{w}, d), (f_{wd}, m_d)).$$

Now we have per-document term counts and max frequency.

Job 3: Document Frequency per Term

Input: $((\mathbf{w}, d), (f_{wd}, m_d))$.

Mapper:

$$((\mathbf{w}, d), (f_{wd}, m_d)) \mapsto (\mathbf{w}, (d, f_{wd}, m_d, 1)).$$

Reducer:

- For each word $\mathbf{w}$, receive $(d, f_{wd}, m_d, 1)$ for all documents containing $\mathbf{w}$.
- Sum all the 1's to get $n_{\mathbf{w}}$, the number of documents containing $\mathbf{w}$.
- Emit for each doc:

$$((\mathbf{w}, d), (f_{wd}, m_d, n_{\mathbf{w}})).$$

Job 4: Compute TF-IDF

Assume N is known.

Input: $((\mathbf{w}, d), (f_{wd}, m_d, n_{\mathbf{w}}))$.

Mapper:

$$\text{TF}_{\mathbf{w}d} = \frac{f_{wd}}{m_d},$$
$$\text{IDF}_{\mathbf{w}} = \log_2 \frac{N}{n_{\mathbf{w}}}.$$

Emit:

$$((\mathbf{w}, d), \text{TF}_{\mathbf{w}d} \cdot \text{IDF}_{\mathbf{w}}).$$

Reducer: Identity (just passes values through).

Scalability Remarks

- Very high-frequency words (e.g. “the”) can cause memory issues when summing n_i .
- Possible solutions:
 - Ignore terms exceeding a frequency threshold.
 - Write intermediate results to disk or use more passes.
- Jobs 3 and 4 can sometimes be combined to save I/O.

1.3 Relational Algebra Operations in MapReduce

We can express standard relational operations (selection, projection, join, etc.) via MapReduce by choosing keys appropriately.

Let R be a relation (table) with tuples t .

Selection $\sigma_C(R)$

Condition $C(t)$ on each tuple.

Map:

$$t \in R \mapsto \begin{cases} (t, t), & \text{if } C(t) \text{ is true,} \\ \text{emit nothing,} & \text{otherwise.} \end{cases}$$

Reduce: Identity. Result set: all selected tuples.

Projection $\pi_S(R)$

We want only attributes in subset S , and we want duplicates removed.

Map:

- For each tuple t , form t' by projecting onto S .
- Emit (t', t') .

Reduce:

$$(t', [t', t', \dots, t']) \mapsto (t', t').$$

Exactly one copy of each distinct t' remains.

Union and Intersection

Assume R and S have the same schema.

Union $R \cup S$: **Map:** For each tuple t in R or S , emit (t, t) . **Reduce:** Identity; we keep one copy of each t .

Intersection $R \cap S$: **Map:** Same as union. **Reduce:**

$$(t, [t, t]) \mapsto (t, t); \quad (t, [t]) \mapsto \text{nothing}.$$

Only tuples that appear in both relations survive.

Difference $R - S$

Map:

- If $t \in R$, emit (t, R) .
- If $t \in S$, emit (t, S) .

Reduce:

$$(t, \text{values}) \mapsto \begin{cases} (t, t), & \text{if values} = [R], \\ \text{nothing,} & \text{otherwise } ([R, S], [S], [S, R]). \end{cases}$$

Natural Join

Example: $R(A, B)$ and $S(B, C)$, joining on attribute B .

Map:

- For each $(a, b) \in R$, emit $(b, (R, a))$.
- For each $(b, c) \in S$, emit $(b, (S, c))$.

Reduce:

- For each key b , collect two lists:

$$\{(R, a_i)\}, \quad \{(S, c_j)\}.$$

- Output all combinations (a_i, b, c_j) .

1.4 Matrix Multiplication in MapReduce

Consider matrices M and N :

$$M \in \mathbb{R}^{I \times J}, \quad N \in \mathbb{R}^{J \times K}, \quad P = MN \in \mathbb{R}^{I \times K}.$$

Elements:

$$p_{ik} = \sum_j m_{ij} n_{jk}.$$

We represent matrices as relations:

$$M(I, J, V) = \{(i, j, m_{ij})\}, \quad N(J, K, W) = \{(j, k, n_{jk})\}.$$

Two-Pass MapReduce Algorithm

First pass: join on j and compute partial products. Map:

- For each (i, j, v) in M , emit $(j, (M, i, v))$.
- For each (j, k, w) in N , emit $(j, (N, k, w))$.

Reduce (per j):

- For each j , obtain:

$$\{(M, i, m_{ij})\}, \quad \{(N, k, n_{jk})\}.$$

- For all i and k pairs, compute $m_{ij} n_{jk}$ and emit

$$((i, k), m_{ij} n_{jk}).$$

Second pass: sum partial products. **Map:** Identity on keys (i, k) :

$$((i, k), v) \mapsto ((i, k), v).$$

Reduce:

$$((i, k), [v_1, v_2, \dots]) \mapsto ((i, k), \sum_q v_q) = ((i, k), p_{ik}).$$

One-Pass Approach (Conceptual)

Emit all necessary keys (i, k) directly from the mapper, so that each reducer can match on j and sum the products. This is more demanding on memory and network but can be done when dimensions are manageable.

1.5 PageRank

PageRank measures the stationary distribution of a random walk on the web graph.

Basic PageRank Without Teleportation

Let M be the transition matrix of the web graph:

- $M_{ij} = 1/\text{outdeg}(j)$ if there is a link from page j to page i .
- Otherwise, $M_{ij} = 0$.

If $v^{(t)}$ is the probability distribution of the surfer at step t , then:

$$v^{(t+1)} = Mv^{(t)}.$$

Under certain conditions (strong connectivity, no dead ends), $v^{(t)}$ converges to a stationary vector v satisfying:

$$v = Mv.$$

Problems: Dead Ends & Spider Traps

- **Dead end:** A page with no outgoing links $\Rightarrow$ mass “leaks”.
- **Spider trap:** A subset of pages that only link among themselves $\Rightarrow$ mass gets stuck.

Taxation / Teleportation

Solution: With probability β follow a link, with probability $1 - \beta$ teleport to a random page.

Let e be the vector of all ones, and n the number of pages. The PageRank iteration becomes:

$$v^{(t+1)} = \beta Mv^{(t)} + (1 - \beta) \frac{1}{n} e.$$

This always converges to a unique stationary distribution.

MapReduce Implementation Outline

Phase 1: Build graph representation Input: (URL, HTML). Mapper:

- Parse outlinks (list of URLs).
- Initialize rank $PR_{\text{init}} = 1/n$.
- Emit (URL, (PR_{init} , outlink-list)).

Reducer: identity.

Phase 2: Iterative PageRank In each iteration:

Mapper: For each page:

$$(\text{URL}, (\text{PR}, \text{outlinks})) \mapsto \begin{cases} (u, \text{PR}/|\text{outlinks}|) & \text{for each } u \text{ in outlinks,} \\ (\text{URL}, \text{outlinks}) & \text{to carry adjacency.} \end{cases}$$

Reducer:

- Sum rank contributions s_{URL} from in-links.
- Apply taxation:

$$PR_{\text{new}}(\text{URL}) = (1 - \beta) \frac{1}{n} + \beta s_{\text{URL}}.$$

- Emit (URL, (PR_{new} , outlinks)).

Iterate until convergence (e.g., 50–75 iterations on the web).

2 Penalty-Based Methods and ADMM

We consider linear models with penalties to control complexity, and we use ADMM to solve large-scale versions.

2.1 Ridge Regression (L2 Penalty)

Model: $y = X\beta + \varepsilon$, where $X \in \mathbb{R}^{N \times p}$, $y \in \mathbb{R}^N$.

Penalized objective:

$$\text{RSS}_\lambda(\beta) = \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

Closed-form solution:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

Ridge shrinks coefficients towards zero but does not set them exactly to zero. It reduces variance and stabilizes estimates when predictors are correlated.

2.2 LASSO (L1 Penalty)

Objective:

$$\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1.$$

- Encourages sparsity: many coefficients are exactly zero.
- Geometrically, the ℓ_1 penalty has diamond-shaped level sets, which intersect quadratic loss contours at coordinate axes.

2.3 Penalized Logistic Regression

Binary response $Y \in \{0, 1\}$, features $x_i \in \mathbb{R}^p$.

Model:

$$\mathbb{P}(Y_i = 1 \mid x_i) = \pi_i = \sigma(x_i^T \beta),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid.

Log-likelihood:

$$\ell(\beta) = \sum_{i=1}^N [y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i)].$$

Penalized objective (to be *maximized* or equivalently *minimize* minus log-likelihood):

$$\max_{\beta} \ell(\beta) - r(\beta) \quad \Leftrightarrow \quad \min_{\beta} -\ell(\beta) + r(\beta),$$

where $r(\beta)$ can be $\lambda \|\beta\|_2^2$, $\lambda \|\beta\|_1$, or a custom penalty.

2.4 ADMM: Alternating Direction Method of Multipliers

We want to solve a constrained convex problem of the form:

$$\min_{x, z} f(x) + g(z) \quad \text{s.t.} \quad Ax + Bz = c.$$

Augmented Lagrangian:

$$L_{\rho}(x, z, y) = f(x) + g(z) + y^T (Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|_2^2,$$

where y is the dual variable and $\rho > 0$ is a penalty parameter.

ADMM Iterations

1. **x-update:**

$$x^{k+1} = \arg \min_x L_{\rho}(x, z^k, y^k).$$

2. **z-update:**

$$z^{k+1} = \arg \min_z L_{\rho}(x^{k+1}, z, y^k).$$

3. Dual update:

$$y^{k+1} = y^k + \rho(Ax^{k+1} + Bz^{k+1} - c).$$

Scaled form: use $u = \frac{1}{\rho}y$ and rewrite updates to improve numerical stability.

2.5 Consensus Optimization via ADMM

We want to minimize a sum of functions:

$$\min_{\beta} \sum_{i=1}^N f_i(\beta),$$

where f_i is the loss for partition i of the data.

Introduce local copies β_i and a global consensus variable β :

$$\min_{\beta_1, \dots, \beta_N, \beta} \sum_{i=1}^N f_i(\beta_i) + r(\beta) \quad \text{s.t.} \quad \beta_i = \beta \quad \forall i.$$

This is in ADMM form:

$$f(\{\beta_i\}) = \sum_i f_i(\beta_i), \quad g(\beta) = r(\beta),$$

with constraints $\beta_i - \beta = 0$.

ADMM Updates for Consensus

Let u_i be scaled dual variables.

1. Local update (per partition):

$$\beta_i^{k+1} = \arg \min_{\beta_i} \left[f_i(\beta_i) + \frac{\rho}{2} \|\beta_i - \beta^k + u_i^k\|_2^2 \right].$$

This is typically a penalized regression or logistic regression using only data from partition i , shrunk toward $(\beta^k - u_i^k)$.

2. Consensus update: Define

$$\bar{\beta}^{k+1} = \frac{1}{N} \sum_{i=1}^N (\beta_i^{k+1} + u_i^k).$$

Then

$$\beta^{k+1} = \arg \min_{\beta} \left[r(\beta) + \frac{N\rho}{2} \|\beta - \bar{\beta}^{k+1}\|_2^2 \right].$$

- If $r(\beta) = 0$, then $\beta^{k+1} = \bar{\beta}^{k+1}$.
- If $r(\beta) = \lambda \|\beta\|_2^2$ (ridge), then this is a quadratic minimization with closed form.
- If $r(\beta) = \lambda \|\beta\|_1$ (lasso), then this is a soft-thresholding step applied to $\bar{\beta}^{k+1}$.

3. Dual update:

$$u_i^{k+1} = u_i^k + \left(\beta_i^{k+1} - \beta^{k+1} \right).$$

Large-Scale Logistic Regression via ADMM

- Split data into K partitions: (X_i, y_i) .
- Local objective on partition i :

$$f_i(\beta_i) = -\ell_i(\beta_i) = - \sum_{j \in \text{part } i} [y_j \log \pi_j + (1 - y_j) \log(1 - \pi_j)],$$

where $\pi_j = \sigma(x_j^T \beta_i)$.

- ADMM coordinates local logistic regressions through consensus β .

This allows logistic regression on massive data using parallel computing.

2.6 ADMM for Logistic Regression with a Custom Penalty Term

The exam may explicitly ask: “Write out the ADMM algorithm for logistic regression with an $L1$, $L2$, or custom penalty term.” This section gives the general template plus common special cases.

Problem Setup

We have data split across K partitions:

$$\{(X_i, y_i)\}_{i=1}^K.$$

On partition i :

$$\ell_i(\beta_i) = - \sum_{j \in \text{part } i} [y_j \log \pi_j + (1 - y_j) \log(1 - \pi_j)], \quad \pi_j = \sigma(x_j^T \beta_i).$$

We want to solve the penalized logistic regression:

$$\min_{\beta} \sum_{i=1}^K \ell_i(\beta) + r(\beta),$$

where $r(\beta)$ is a *custom penalty* term (can be $L1$, $L2$, elastic net, group penalty, etc).

Consensus Formulation

Introduce local copies β_i and a global variable β :

$$\min_{\{\beta_i\}, \beta} \sum_{i=1}^K \ell_i(\beta_i) + r(\beta) \quad \text{s.t. } \beta_i = \beta, \quad i = 1, \dots, K.$$

This is exactly in the consensus ADMM form.

Augmented Lagrangian (Scaled Form)

Introduce scaled dual variables u_i and penalty parameter $\rho > 0$. The scaled-form augmented Lagrangian is:

$$L_\rho(\{\beta_i\}, \beta, \{u_i\}) = \sum_{i=1}^K \left[\ell_i(\beta_i) + \frac{\rho}{2} \|\beta_i - \beta + u_i\|_2^2 \right] + r(\beta) + \text{const.}$$

ADMM alternates between updating local β_i , global β , and dual u_i .

ADMM Updates (General Custom Penalty)

1. Local logistic regression update (per partition i):

$$\beta_i^{k+1} = \arg \min_{\beta_i} \left[\ell_i(\beta_i) + \frac{\rho}{2} \|\beta_i - \beta^k + u_i^k\|_2^2 \right].$$

Interpretation:

- This is a logistic regression problem on partition i , with an additional quadratic penalty that shrinks β_i toward $(\beta^k - u_i^k)$.
- In practice, you would solve this with a standard logistic regression solver (e.g. Newton, coordinate descent, or a library like `glmnet`) with an L2 penalty centered at $(\beta^k - u_i^k)$.

2. Consensus update for global β (custom penalty): Compute the average of adjusted local variables:

$$\bar{\beta}^{k+1} = \frac{1}{K} \sum_{i=1}^K (\beta_i^{k+1} + u_i^k).$$

Then solve:

$$\beta^{k+1} = \arg \min_{\beta} \left[r(\beta) + \frac{K\rho}{2} \|\beta - \bar{\beta}^{k+1}\|_2^2 \right].$$

This is the **proximal step** of the custom penalty:

$$\beta^{k+1} = \text{prox}_{\frac{1}{K\rho}r}(\bar{\beta}^{k+1}).$$

3. Dual variable update:

$$u_i^{k+1} = u_i^k + (\beta_i^{k+1} - \beta^{k+1}), \quad i = 1, \dots, K.$$

These three lines are what you should be ready to write in the exam for the general custom-penalty case.

Special Cases: L2, L1, and Other Penalties

(a) L2 penalty (ridge logistic regression):

$$r(\beta) = \lambda \|\beta\|_2^2.$$

Consensus update is:

$$\beta^{k+1} = \arg \min_{\beta} \left[\lambda \|\beta\|_2^2 + \frac{K\rho}{2} \|\beta - \bar{\beta}^{k+1}\|_2^2 \right].$$

This is a quadratic problem with closed form. Setting the gradient to zero:

$$2\lambda\beta + K\rho(\beta - \bar{\beta}^{k+1}) = 0 \quad \Rightarrow \quad (2\lambda + K\rho)\beta = K\rho \bar{\beta}^{k+1}.$$

So:

$$\beta^{k+1} = \frac{K\rho}{2\lambda + K\rho} \bar{\beta}^{k+1}.$$

(b) L1 penalty (lasso logistic regression):

$$r(\beta) = \lambda \|\beta\|_1.$$

Consensus update is:

$$\beta^{k+1} = \arg \min_{\beta} \left[\lambda \|\beta\|_1 + \frac{K\rho}{2} \|\beta - \bar{\beta}^{k+1}\|_2^2 \right].$$

This is elementwise **soft-thresholding**:

$$\beta_j^{k+1} = \mathcal{S}_{\lambda/(K\rho)} \left(\bar{\beta}_j^{k+1} \right),$$

where

$$\mathcal{S}_{\tau}(z) = \text{sign}(z) \max(|z| - \tau, 0)$$

is the soft-thresholding operator.

(c) Elastic net or other separable penalties: For a separable penalty

$$r(\beta) = \sum_j r_j(\beta_j),$$

the consensus step decouples across coordinates:

$$\beta_j^{k+1} = \arg \min_{\beta_j} \left[r_j(\beta_j) + \frac{K\rho}{2} (\beta_j - \bar{\beta}_j^{k+1})^2 \right].$$

Each coordinate update is a 1D proximal step.

(d) **Group penalties (e.g. group lasso):** If

$$r(\beta) = \sum_{g \in \mathcal{G}} \lambda_g \|\beta_g\|_2,$$

where β_g is a block of coefficients, then the consensus step:

$$\min_{\beta} \sum_g \lambda_g \|\beta_g\|_2 + \frac{K\rho}{2} \sum_g \|\beta_g - \bar{\beta}_g^{k+1}\|_2^2$$

decouples over groups. Each group update is a *block soft-thresholding* (a shrinkage in the direction of $\bar{\beta}_g^{k+1}$).

Summary: “Template Answer” for the Exam

If asked to “write out the ADMM algorithm for logistic regression with a custom penalty term”, you can write:

- Problem:

$$\min_{\{\beta_i\}, \beta} \sum_{i=1}^K \ell_i(\beta_i) + r(\beta) \quad \text{s.t. } \beta_i = \beta.$$

- ADMM iterations:

$$\beta_i^{k+1} = \arg \min_{\beta_i} \left[\ell_i(\beta_i) + \frac{\rho}{2} \|\beta_i - \beta^k + u_i^k\|_2^2 \right],$$

$$\bar{\beta}^{k+1} = \frac{1}{K} \sum_{i=1}^K (\beta_i^{k+1} + u_i^k),$$

$$\beta^{k+1} = \arg \min_{\beta} \left[r(\beta) + \frac{K\rho}{2} \|\beta - \bar{\beta}^{k+1}\|_2^2 \right],$$

$$u_i^{k+1} = u_i^k + (\beta_i^{k+1} - \beta^{k+1}).$$

Then briefly state the closed-form solution of the consensus step for L1, L2, or whichever specific penalty the question mentions.

3 Support Vector Machines (SVM)

SVM is a maximum-margin classifier in a (possibly high-dimensional) feature space.

3.1 Linear Separating Hyperplane

Given labeled data (x_i, y_i) with $y_i \in \{-1, +1\}$, a linear classifier is:

$$f(x) = \text{sign}(\beta^T x + b).$$

Hyperplane: $\beta^T x + b = 0$.

3.2 Margin and Optimal Separator

Distance from a point x to the hyperplane is:

$$r = \frac{\beta^T x + b}{\|\beta\|_2}.$$

For a correctly classified point:

$$y_i(\beta^T x_i + b) > 0.$$

Margin ρ is the width between the two class boundaries:

$$\beta^T x + b = 1, \quad \beta^T x + b = -1.$$

Distance between these planes:

$$\rho = \frac{2}{\|\beta\|_2}.$$

Max-margin classifier $\Rightarrow$ minimize $\|\beta\|_2$ (or equivalently $\frac{1}{2}\|\beta\|_2^2$).

3.3 Hard-Margin SVM

Assume data are linearly separable and scaled so the margin boundaries correspond to ± 1 .

Primal problem:

$$\min_{\beta, b} \frac{1}{2} \|\beta\|_2^2 \quad \text{subject to } y_i(\beta^T x_i + b) \geq 1, \quad \forall i.$$

3.4 Soft-Margin SVM

When data is not linearly separable, introduce slack variables $\xi_i \geq 0$:

$$y_i(\beta^T x_i + b) \geq 1 - \xi_i.$$

Objective:

$$\min_{\beta, b, \xi} \frac{1}{2} \|\beta\|_2^2 + C \sum_{i=1}^N \xi_i$$

subject to:

$$y_i(\beta^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

$C > 0$ balances margin size vs. misclassification penalty.

3.5 Dual Formulation and Support Vectors

For the soft-margin SVM, the dual problem is:

$$\max_{\alpha} Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

subject to:

$$\sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

The optimal weight vector is:

$$\beta^* = \sum_{i=1}^N \alpha_i y_i x_i.$$

Support vectors are the points with $\alpha_i > 0$; the solution depends only on them.

The decision function:

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i x_i^T x + b \right).$$

3.6 Nonlinear SVM and the Kernel Trick

We map inputs into a (possibly high-dimensional) feature space via $\phi(x)$ and apply a linear SVM in that space:

$$f(x) = \text{sign}(\beta^T \phi(x) + b).$$

Instead of working explicitly with $\phi(x)$, define a kernel:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j).$$

Dual problem:

$$\max_{\alpha} Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j),$$

subject to:

$$\sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

Decision function:

$$f(x) = \text{sign} \left(\sum_{i=1}^N \alpha_i y_i K(x_i, x) + b \right).$$

Common Kernels

- Linear: $K(x, x') = x^T x'$.
- Polynomial: $K(x, x') = (1 + x^T x')^p$.
- Gaussian RBF: $K(x, x') = \exp \left(-\frac{\|x - x'\|^2}{2\sigma^2} \right)$.

4 Random Forests

Random forests are ensembles of decision trees.

4.1 Decision Trees (CART)

Decision trees recursively partition the feature space into rectangular regions.

For regression:

- At each node, choose a variable j and split point s .
- Define regions:

$$R_1(j, s) = \{x : x_j < s\}, \quad R_2(j, s) = \{x : x_j \geq s\}.$$

- Choose (j, s) to minimize sum of squared errors:

$$\sum_{x_i \in R_1(j, s)} (y_i - \bar{y}_{R_1})^2 + \sum_{x_i \in R_2(j, s)} (y_i - \bar{y}_{R_2})^2.$$

For classification:

- Use impurity measures such as Gini index or deviance at each node.
- Choose split that most reduces impurity.

Greedy, top-down procedure; pruning can be used to avoid overfitting.

4.2 Random Forest Algorithm

Random forest for classification or regression:

1. For $b = 1$ to B (number of trees):
 - (a) Draw a bootstrap sample (with replacement) from the training data.
 - (b) Grow a tree T_b :
 - At each node, randomly select m predictors out of p .
 - Find the best split among those m predictors.
 - Continue splitting until some stopping criterion (e.g. minimum node size) is reached; no pruning.
2. For prediction:
 - Regression: average the predictions from all trees.
 - Classification: majority vote over trees.

Key ideas:

- **Bagging** reduces variance by averaging over bootstrap samples.
- Random feature selection at each split reduces correlation among trees.

4.3 Out-of-Bag (OOB) Error

Because each tree trains on a bootstrap sample, about 1/3 of the observations are *out-of-bag* for that tree.

- For each observation, aggregate predictions from all trees where it was OOB.
- Compare aggregated OOB prediction to true label.
- OOB error is an unbiased estimate of test error.

4.4 Variable Importance in Random Forests

Permutation importance:

1. Compute baseline OOB error using the original data.
2. For each predictor x_p :
 - (a) Randomly permute x_p among the training samples to create D' .
 - (b) Recompute OOB error using D' .
 - (c) The increase in OOB error is the importance of x_p .

4.5 Random Forests for Big Data

- Data can be very large and distributed across a cluster (e.g. HDFS).
- Each tree can be trained on a subset of the data, possibly on a separate node.
- Ensembles of many smaller trees can outperform a single tree trained on all data.
- Training and prediction are easily parallelizable.